

Fundamentals of Deep Learning and Programming

Lecture 1: Useful Python Libraries

Fall Semester 2025

Adjunct Professor: Donghoon Kang

- In Python, functions are callable (you can do `f(x)`).
- Objects (instances of classes) are **not** callable by default.
- But if a class defines the special method `__call__`, then **its instances become callable, like functions.**

```
class MyAdder:
    def __init__(self, n):
        self.n = n

    def __call__(self, x):
        return x + self.n

adder = MyAdder(10)  # adder is an instance of MyAdder class
print(adder(5))      # 15 (calls adder.__call__(5))
```

15

- Because we defined `__call__`, we can use `adder(5)` as if `adder` were a function.
- Internally, Python executes `adder.__call__(5)`.

- It lets you write objects that **behave like functions**, **but also carry internal state**.
- This is exactly how PyTorch's `nn.Module` works.

- Inside `nn.Module.__call__`, PyTorch does extra work:
 - Calls `.forward()`
 - Applies post-processing

So you can write `criterion(y_hat, y)` instead of `criterion.forward(y_hat, y)`
– much cleaner!

Numpy

= Numerical Python

`conda install numpy`

`pip install numpy`



- **Multi-dimensional arrays (`ndarray`)** : Unlike Python lists, NumPy arrays store elements of the same type, allowing faster computation and less memory usage.
- **Mathematical operations**: A wide range of high-performance functions for linear algebra, statistics, and random number generation.
- **Vectorization**: Applying operations to **entire arrays at once**, making code more concise and much **faster**.
- **Easy Integration with other Python libraries** such as pandas, SciPy, scikit-learn, TensorFlow, and PyTorch.

performing **operations on entire arrays (vectors, matrices, tensors)** at once, instead of using explicit Python for loops.

```
import numpy as np

# Create arrays
a = np.array([1, 2, 3])
b = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a)           # [1 2 3]
print(b)           # 2 rows, 3 cols

print(a.ndim)      # number of axes: 1
print(b.ndim)      # number of axes: 2
print(b.shape)     # (2, 3) -> 2 rows, 3 columns
print(b.dtype)     # data type (e.g., int64)

# Make arrays of zeros/ones or a range
z = np.zeros((2, 3))      # shape 2x3, all zeros (float)
o = np.ones((3, 2), dtype=int) # shape 3x2, all ones (int)
r = np.arange(0, 10, 2)   # [0 2 4 6 8]
lin = np.linspace(0, 1, 5) # [0. 0.25 0.5 0.75 1. ]
print(z, o, r, lin, sep="\n")
```

```
[1 2 3]
[[1 2 3]
 [4 5 6]]
1
2
(2, 3)
int64
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
[[1 1]
 [1 1]
 [1 1]]
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1. ]
```

c02_numpy01.py

Numpy

Indexing & Slicing

```
B = [[10, 11, 12],  
      [20, 21, 22],  
      [30, 31, 32]]
```

list

```
print(B[0][1])
```

11



```
import numpy as np
```

numpy

```
A = np.array([[10, 11, 12],  
              [20, 21, 22],  
              [30, 31, 32]])
```

```
print(A[0, 1])    # row 0, col 1 -> 11  
print(A[1])      # entire row 1 -> [20 21 22]  
print(A[:, 2])   # entire col 2 -> [12 22 32]  
print(A[0:2, 1:3]) # rows 0..1, cols 1..2 -> [[11 12],[21 22]]
```

```
11  
[20 21 22]  
[12 22 32]  
[[11 12]  
 [21 22]]
```

```
# Boolean indexing (filtering)  
mask = (A % 20 == 0)    # True where divisible by 20  
print(mask)             # [[False False False], [ True False False], [ True False False]]  
print(A[mask])          # [20]
```

```
[[False False False]  
 [ True False False]  
 [False False False]  
 [20]]
```

c02_numpy02.py

performing **operations on entire arrays (vectors, matrices, tensors) at once**, instead of using explicit Python for loops.



```
a = list(range(5))  
b = list(range(5))
```

list, for loop

```
result = [a[i] + b[i] for i in range(len(a))] # without vectorization
```

[0, 2, 4, 6, 8]

```
import numpy as np  
a = np.arange(5)  
b = np.arange(5)
```

numpy

```
result = a + b # vectorization
```

[0 2 4 6 8]

c02_numpy03.py



```
numbers = list(range(5))  
squared = [x**2 for x in  
numbers]  
print(squared)
```

list, for loop

[0, 1, 4, 9, 16]

```
numbers = np.arange(5)  
squared = numbers ** 2 #  
vectorization  
print(squared)
```

numpy

[0 1 4 9 16]

Numpy

Broadcasting

(different shapes that “fit”)

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)

d = 10
print(A + d)
# [[11 12 13]
#  [14 15 16]]

print(type(d)) # <class 'int'>
print(type(A+d)) # <class 'numpy.ndarray'>
```

```
[[11 22 33]
 [14 25 36]]
```

```
<class 'int'>
<class 'numpy.ndarray'>
```

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)
b = np.array([10, 20, 30]) # shape (3,)

print(A + b)
# [[11 22 33]
#  [14 25 36]]
```

```
[[11 12 13]
 [14 15 16]]
```

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)
c = np.array([[100],
              [200]]) # shape (2,1)

print(A + c)
# [[101 102 103]
#  [204 205 206]]
```

```
[[101 102 103]
 [204 205 206]]
```

c02_numpy04.py

Numpy

Axis

axis=0  go down columns (across rows)
axis=1  go across rows (across columns)

IMPORTANT



```
import numpy as np
```

```
B = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

```
print(B.sum()) # 21
```

```
print(B.mean(), B.min(), B.max()) # 3.5 1 6
```

```
print(np.median(B), np.std(B)) # 3.5, standard deviation
```

```
# axis -----
```

```
print(B.sum(axis=0)) # column sums -> [5 7 9]
```

```
print(B.sum(axis=1)) # row sums -> [ 6 15]
```

```
# column-wise mean (axis=0)
```

```
col_means = B.mean(axis=0)
```

```
print("axis=0 (columns) mean:", col_means) # [2.5 3.5 4.5]
```

```
# row-wise mean (axis=1)
```

```
row_means = B.mean(axis=1)
```

```
print("axis=1 (rows) mean:", row_means) # [2. 5.]
```

21

3.5 1 6

3.5 1.707825127659933

[5 7 9]

[6 15]

axis=0 (columns) mean: [2.5 3.5 4.5]

axis=1 (rows) mean: [2. 5.]

c02_numpy05.py

Numpy

reshape(), ravel()

```
import numpy as np

C = np.arange(12)           # [0..11]
print(C.reshape(3, 4))     # shape (3,4)
print(C.reshape(-1, 6))    # -1 lets NumPy infer -> (2,6)
print(C.ravel())           # flat view if possible
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

c02_numpy06.py

Numpy

`np.array([1., 2.])` is a 1-D array whose shape is (2,).

`b = np.array([1., 2.])` is neither a row nor a column vector. It's a 1-D array with shape (2,). In NumPy, a vector's "direction" only becomes clear when you make it 2-D array.

```
import numpy as np

A = np.array([[1., 2.],
              [3., 4.]])

row = b[None, :]          # shape: (1, 2) ← row vector
col = b[:, None]          # shape: (2, 1) ← column vector

# Equivalent forms
row2 = b.reshape(1, -1)   # (1, 2)
col2 = b.reshape(-1, 1)   # (2, 1)

# Matrix multiplication
print(b @ A)              # (2,) @ (2,2) → (2,)   (behaves like a row in this context)
print(A @ b)              # (2,2) @ (2,) → (2,)   (behaves like a column in this context)

print(row @ A)            # (1,2) @ (2,2) → (1,2)
print(A @ col)            # (2,2) @ (2,1) → (2,1)
```

`row = b[None, :]`

`None` (= `np.newaxis`) tells NumPy to 'add a new axis (dimension).'

`[ 7. 10.]`

`[ 5. 11.]`

`[[ 7. 10.]]`

`[[ 5.]`
`[11.]]`



c02_numpy06.py

Numpy

np.stack()

np.stack() → adds a new axis.

axis=0 ↓ go down columns (across rows)
axis=1 → go across rows (across columns)

```
import numpy as np

X = np.array([1, 2, 3])
Y = np.array([4, 5, 6])

print(np.stack([X, Y], axis=0)) # [[1 2 3], [4 5 6]]
print(np.stack([X, Y], axis=1)) # [[1 4], [2 5], [3 6]]

print(np.vstack([X, Y])) # [[1 2 3], [4 5 6]]
print(np.hstack([X, Y])) # [1 2 3 4 5 6]
```

```
import numpy as np

X = np.array([1, 2, 3])
Y = np.array([4, 5, 6])

print(X.shape, Y.shape)

a = np.stack([X, Y], axis=0)
b = np.stack([X, Y], axis=1)
print(a.shape)
print(b.shape)
```

c02_numpy06.py

```
import numpy as np
```

```
A = np.array([[2., 1.],  
              [1., 3.]])  
b = np.array([1., 2.])
```

```
# Solve  $Ax = b$ 
```

```
x = np.linalg.solve(A, b)  
print("solution:", x)
```

```
solution: [0.2 0.6]
```

```
# Matrix multiply (dot product)
```

```
M = np.array([[1, 2],  
              [3, 4]])  
N = np.array([[10, 20],  
              [30, 40]])
```

```
print(M * N)          # caution: element-wise multiplication  
print(M @ N)          # matrix multiplication  
print(np.dot(M, N))   # matrix multiplication (M @ N)
```

```
print(np.linalg.eig(M)) # eigenvalues & eigenvectors
```

```
EigResult(eigenvalues=array([-0.37228132,  5.37228132]), eigenvectors=array([[ -0.82456484, -0.41597356],  
 [ 0.56576746, -0.90937671]]))
```

c02_numpy06.py

```
[[ 10  40]  
 [ 90 160]]
```

```
[[ 70 100]  
 [150 220]]
```

```
[[ 70 100]  
 [150 220]]
```

Pandas

= Spreadsheet using Python

`conda install pandas`

`pip install pandas`

- **Structured data**: Works perfectly with tabular data (rows & columns). Similar to Excel, but much more powerful.
- **Fast computations** with the flexibility of Python.
- Widely used in **data science, machine learning**, and finance.



Pandas

Series

A **1D** labeled **array**

- A Series is one-dimensional, but **pandas adds labels (index) to each value.**
- This makes the data more powerful and easier to use.

Series

A	10
B	20
C	30
D	40

index

value

```
import numpy as np
import pandas as pd

# Create a Series from a NumPy array
data = np.array([10, 20, 30, 40])
s = pd.Series(data, index=["A", "B", "C", "D"])
print(s)

print(s.values) # If you want to see values
```

```
A    10
B    20
C    30
D    40
dtype: int64
```

```
[10 20 30 40]
```

c02_pandas01.py

When you print a Series, pandas shows a summary of the object:

- **Index (labels)** → on the left (A, B, C, D)
- **Values** → on the right (10, 20, 30, 40)
- **dtype** → at the bottom (int64)

Pandas

DataFrame

A **2D** labeled table

Category	Age	weight
dog	3	15
cat	2	10
goat	8	35

```
import pandas as pd

# Create a DataFrame from a dictionary
data = {
    "Category": ["dog", "cat", "goat"],
    "Age": [3, 2, 8],
    "weight": [15, 10, 35]
}

%converts this dictionary into a DataFrame (a table)
df = pd.DataFrame(data)
print(df)
```

```
print(df["Age"])
```

```
print(df["Age"].mean()) # Average of "Age"
```

```
4.333333333333333
```

- **Index** → labels for rows (here: 0, 1, 2).
- **Series** → each individual column of the DataFrame.
- **DataFrame** → a collection of multiple Series sharing the same index.

	Category	Age	weight
0	dog	3	15
1	cat	2	10
2	goat	8	35

0	3
1	2
2	8

Name: Age, dtype: int64

Pandas

Category	Age	weight
dog	3	15
cat	2	10
goat	8	35

```
import pandas as pd

# Create a DataFrame from a dictionary
data = {
    "Category": ["dog", "cat", "goat"],
    "Age": [3, 2, 8],
    "weight": [15, 10, 35]
}
df = pd.DataFrame(data)
```

```
print(df.iloc[0]) # first row
```

```
print(df.iloc[1]) # second row
```

```
print(df.iloc[:,0]) # first column
```

- **iloc** stands for integer-location based indexing

c02_pandas01.py

```
Category    dog
Age          3
weight      15
Name: 0, dtype: object
```

```
Category    cat
Age          2
weight      10
Name: 1, dtype: object
```

```
0    dog
1    cat
2    goat
Name: Category, dtype: object
```

Matplotlib

= making charts and plots

`conda install matplotlib`

`pip install matplotlib`

- You can make **line charts, bar charts, scatter plots, histograms, and more.**



Matplotlib

Draw a Line

```
import numpy as np
import matplotlib.pyplot as plt

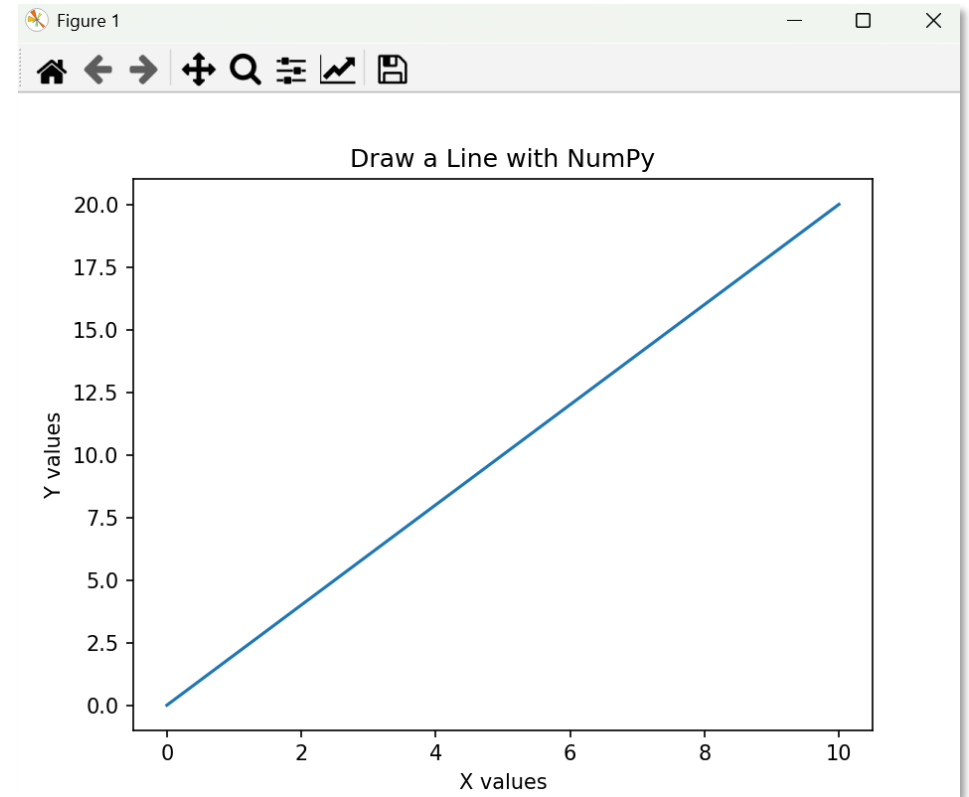
# Create data with NumPy
x = np.linspace(0, 10, 100)    # 100 points from 0 to 10
y = 2 * x                     # y = 2x

plt.plot(x, y)

plt.title("Draw a Line with NumPy")
plt.xlabel("X values")
plt.ylabel("Y values")

plt.show()
```

c02_matplotlib01.py



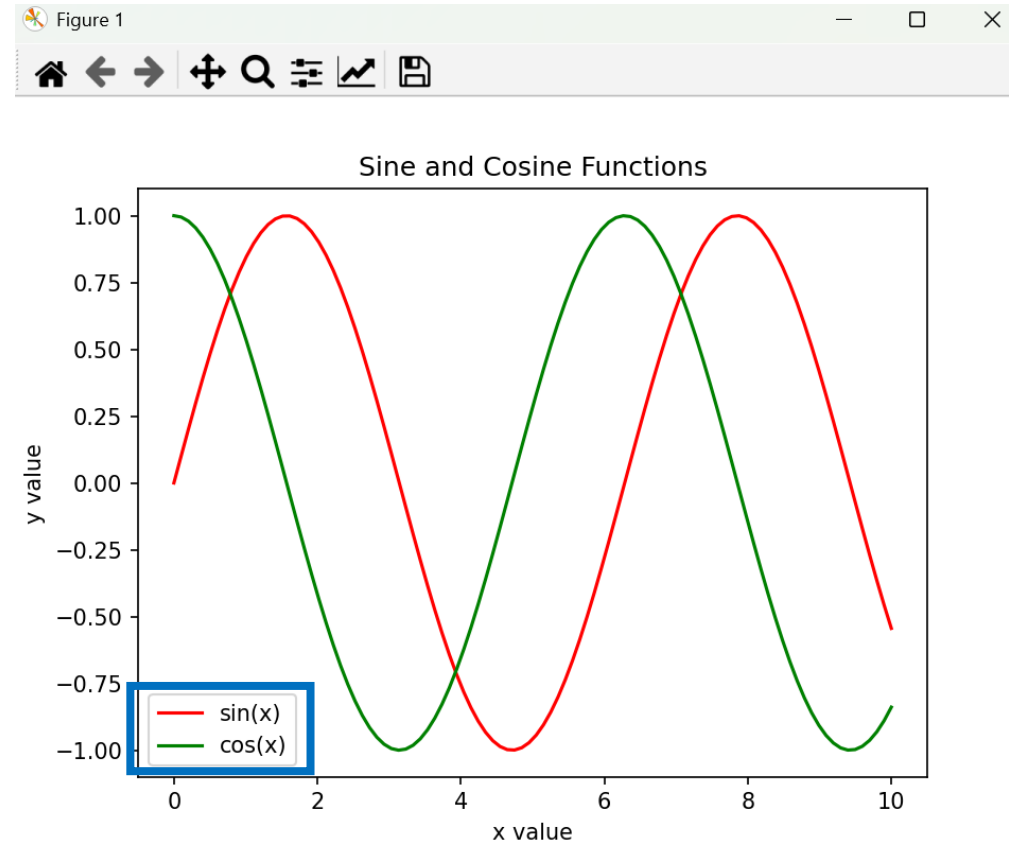
```
import numpy as np
import matplotlib.pyplot as plt

# Create data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Plot both
plt.plot(x, y1, label="sin(x)", color="red")
plt.plot(x, y2, label="cos(x)", color="green")

plt.xlabel("x value")
plt.ylabel("y value")
plt.title("Sine and Cosine Functions")
plt.legend()
plt.show()
```

c02_matplotlib02.py



Matplotlib

Multiple Graphs Side by Side

```
import numpy as np
import matplotlib.pyplot as plt

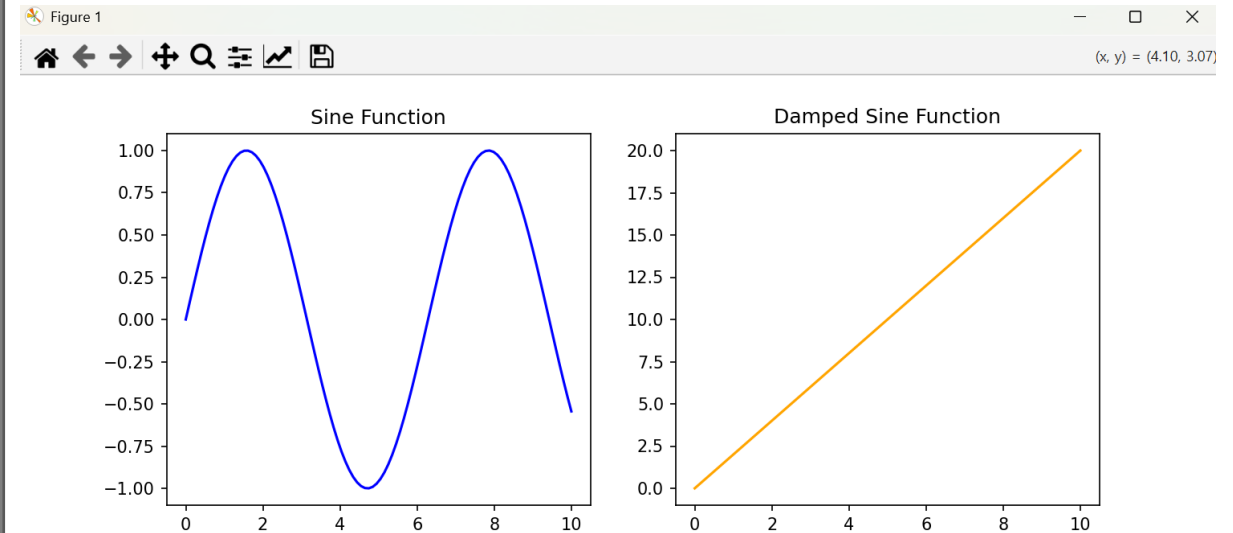
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = 2 * x

# fig = whole figure (canvas),
# axes = subplots (small plots inside the figure)
# 1 row, 2 columns → two plots side by side
# figsize=(10, 4) → width=10 inches, height=4 inches
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Left subplot
axes[0].plot(x, y1, color="blue")
axes[0].set_title("Sine Function")
axes[0].set_xlabel("x")
axes[0].set_ylabel("sin(x)")

# Right subplot
axes[1].plot(x, y2, color="red")
axes[1].set_title("Linear Function")
axes[1].set_xlabel("x")
axes[1].set_ylabel("2*x")

plt.tight_layout() # auto-adjust spacing between
                  # subplots
plt.show()
```



Matplotlib

```
import matplotlib.pyplot as plt
import numpy as np
```

Sample data

```
x = np.arange(5)
y = np.random.randint(1, 10, size=5)
scatter_x = np.random.rand(50)
scatter_y = np.random.rand(50)
hist_data = np.random.randn(1000)
```

Create figure with 3 subplots

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
```

Bar chart

```
axes[0].bar(x, y, color='skyblue')
axes[0].set_title("Bar Chart")
```

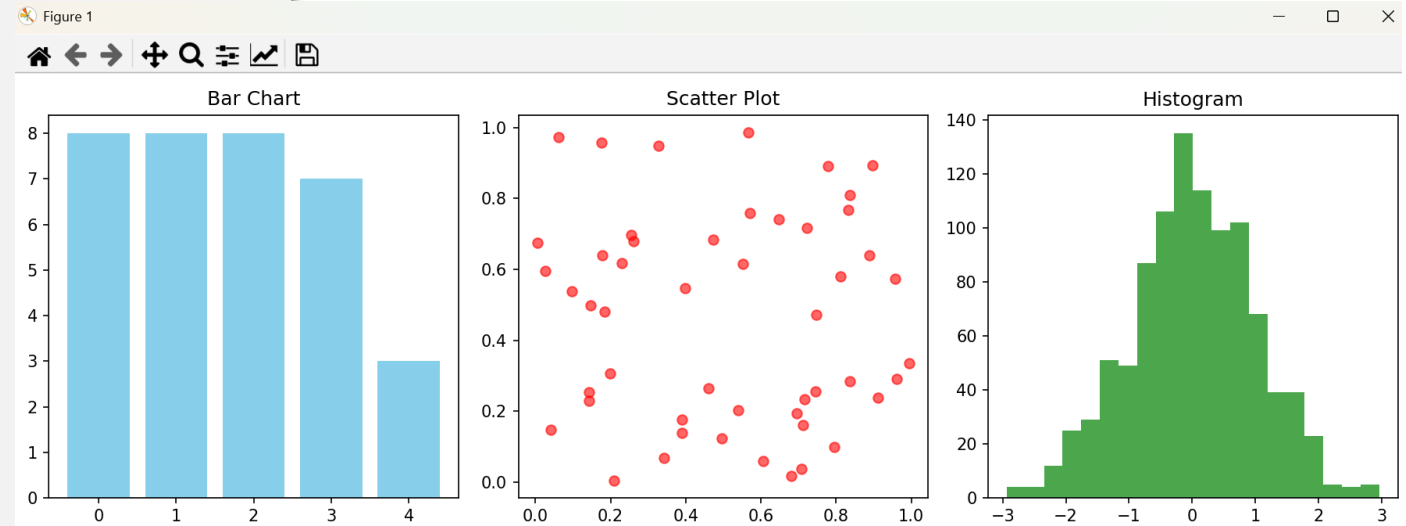
Scatter plot

```
axes[1].scatter(scatter_x, scatter_y, c='red', alpha=0.6)
axes[1].set_title("Scatter Plot")
```

Histogram

```
axes[2].hist(hist_data, bins=20, color='green', alpha=0.7)
axes[2].set_title("Histogram")
```

```
plt.tight_layout()
plt.show()
```



creates a bar chart, scatter plot, and histogram



Basics

Epoch, Batch, Iteration



One **Epoch**

One **full training cycle** over the entire dataset



Batch

(=mini-batch) A subset of the dataset used in one update step



Iteration

Processing one batch

Example

Suppose we have a dataset of **1,000 samples** and we choose a **batch size of 100**.

- **Batch size = 100** → Each batch contains 100 samples.
- Total dataset = 1,000 samples → That means there are 10 batches per epoch.
- So, **1 epoch = 10 iterations**.

If we train the model for **5 epochs**:

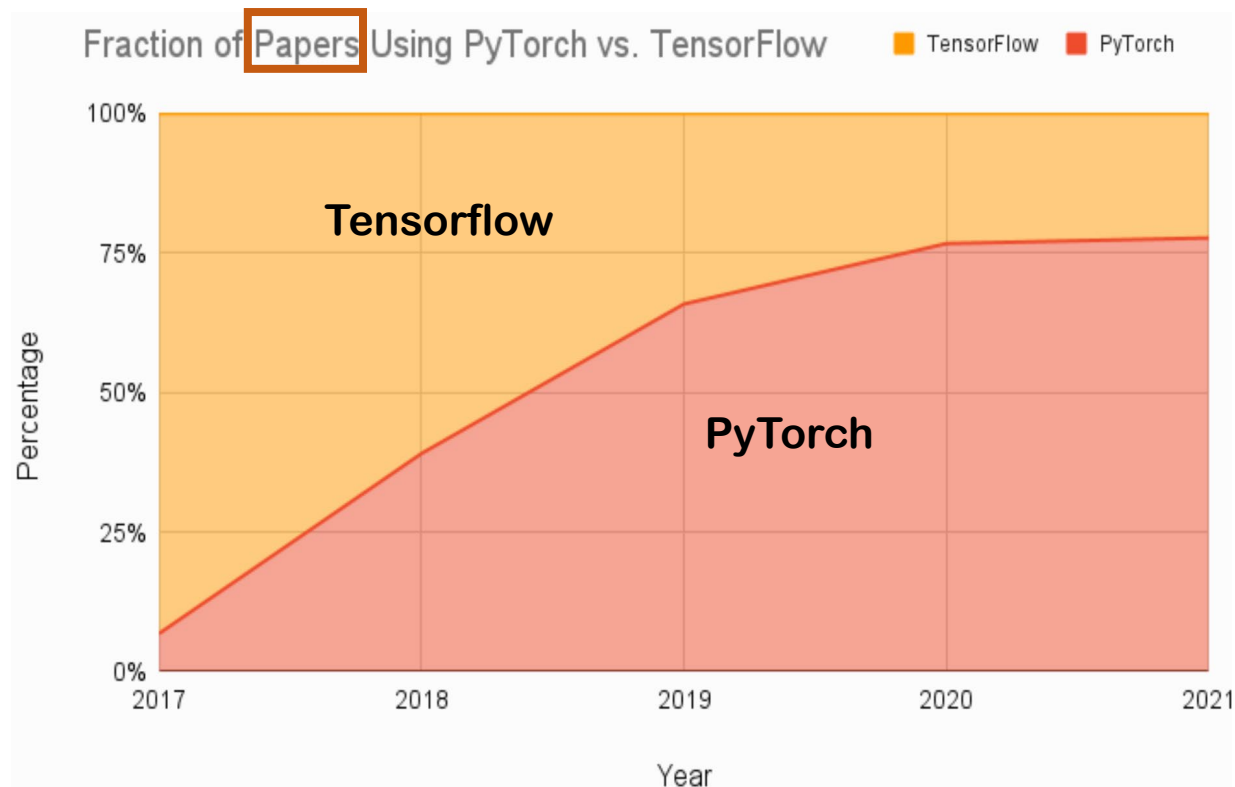
The model will see the entire dataset 5 times.

Total iterations = **5 epochs** × 10 iterations/epoch = 50 iterations.



PyTorch Basics

PyTorch vs. TensorFlow



Creating Tensor

PyTorch

A PyTorch **tensor** is a multi-dimensional **array** that is optimized for numerical computation.

```
import torch
```

```
# Create tensors
```

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])
```

```
print(a)                # tensor([1, 2, 3])
print(b)                # 2 rows, 3 cols
```

```
print(a.ndim)           # number of axes: 1
print(b.ndim)           # number of axes: 2
print(b.shape)          # torch.Size([2, 3]) -> 2 rows, 3 columns
print(b.dtype)          # data type (e.g., torch.int64)
```

```
# Make tensors of zeros/ones or a range
```

```
z = torch.zeros((2, 3))          # shape 2x3, all zeros (float)
o = torch.ones((3, 2), dtype=torch.int32) # shape 3x2, all ones (int)
r = torch.arange(0, 10, 2)        # [0, 2, 4, 6, 8]
lin = torch.linspace(0, 1, steps=5) # [0., 0.25, 0.5, 0.75, 1.]
print(z, o, r, lin, sep="\n")
```

```
tensor([1, 2, 3])
tensor([[1, 2, 3],
        [4, 5, 6]])
1
2
torch.Size([2, 3])
torch.int64
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[1, 1],
        [1, 1]], dtype=torch.int32)
tensor([0, 2, 4, 6, 8])
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

c02_pytorch01.py

1. Similar to NumPy arrays – they support **efficient numerical operations**.
2. **Automatic differentiation** – when used with `requires_grad=True`, tensors can **track computations for backpropagation in deep learning**.
3. **GPU acceleration** – tensors can be **easily moved to GPUs** (`.to("cuda")`) for faster computation.
4. Versatility – they can represent almost any type of data: text embeddings, image pixels, audio signals, etc.

Indexing & Slicing

PyTorch

```
import torch

A = torch.tensor([[10, 11, 12],
                  [20, 21, 22],
                  [30, 31, 32]])

print(A[0, 1].item()) # row 0, col 1 -> 11
print(A[1])           # entire row 1 -> tensor([20, 21, 22])
print(A[:, 2])         # entire col 2 -> tensor([12, 22, 32])
print(A[0:2, 1:3])     # rows 0..1, cols 1..2
                      # -> tensor([[11, 12], [21, 22]])

# Boolean indexing (filtering)
mask = (A % 20 == 0)   # True where divisible by 20
print(mask)
print(A[mask])          # tensor([20])
```

c02_pytorch02.py

```
tensor([[False, False, False],
        [ True, False, False],
        [False, False, False]])
tensor([20])
```

Numpy

```
import numpy as np

A = np.array([[10, 11, 12],
              [20, 21, 22],
              [30, 31, 32]])

print(A[0, 1])         # row 0, col 1 -> 11
print(A[1])            # entire row 1 -> [20 21 22]
print(A[:, 2])          # entire col 2 -> [12 22 32]
print(A[0:2, 1:3])     # rows 0..1, cols 1..2
                      # -> [[11 12], [21 22]]

# Boolean indexing (filtering)
mask = (A % 20 == 0)   # True where divisible by 20
print(mask)
print(A[mask])          # [20]
```

```
[[False False False]
 [ True False False]
 [False False False]]
[20]
```

Vectorization

performing operations on entire arrays (vectors, matrices, tensors) at once, instead of using explicit Python for loops.

PyTorch

```
import torch

a = torch.arange(5) # tensor([0, 1, 2, 3, 4])
b = torch.arange(5) # tensor([0, 1, 2, 3, 4])

result = a + b # vectorization
print(result) # tensor([0, 2, 4, 6, 8])

numbers = torch.arange(5) # tensor([ 0, 1, 4, 9, 16])
squared = numbers ** 2 # vectorization
print(squared)
```

c02_pytorch03.py

Numpy

```
import numpy as np

a = np.arange(5)
b = np.arange(5)

result = a + b # vectorization
print(result) # [0 2 4 6 8]

numbers = np.arange(5)
squared = numbers ** 2 # vectorization
print(squared) # [ 0 1 4 9 16]
```

list, for loop

```
a = list(range(5))
b = list(range(5))

result = [a[i] + b[i] for i in range(len(a))]
# without vectorization
# [0, 2, 4, 6, 8]
```

```
numbers = list(range(5))
squared = [x**2 for x in numbers]
# without vectorization

print(squared) # [0, 1, 4, 9, 16]
```

Broadcasting

(different shapes that “fit”)

PyTorch

```
import torch

A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]]) # shape (2,3)

d = 10
print(A + d) # tensor([[11, 12, 13],
#                  [14, 15, 16]])

print(type(d)) # <class 'int'>
print(type(A + d)) # <class 'torch.Tensor'>
```

```
import torch

A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]]) # shape (2,3)
b = torch.tensor([10, 20, 30]) # shape (3,)

print(A + b)
# tensor([[11, 22, 33],
#         [14, 25, 36]])
```

Numpy

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)

d = 10
print(A + d) # [[11 22 33]
#             [14 25 36]]

print(type(d)) # <class 'int'>
print(type(A + d)) # <class 'numpy.ndarray'>
```

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)
b = np.array([10, 20, 30]) # shape (3,)

print(A + b)
# [[11 22 33]
#   [14 25 36]]
```

```

import torch

B = torch.tensor([[1, 2, 3],
                  [4, 5, 6]], dtype=torch.float32)
# set dtype=float for convenient mean calculation

print(B.sum())           # tensor(21.)
print(B.mean(), B.min(), B.max())
# tensor(3.5) tensor(1.) tensor(6.)

print(B.median(), B.std()) # tensor(3.5), tensor(1.8708)

# dim in PyTorch -----
print(B.sum(dim=0))      # column sums -> tensor([5., 7., 9.])
print(B.sum(dim=1))      # row sums    -> tensor([ 6., 15.])

# column-wise mean (dim=0)
col_means = B.mean(dim=0)
print("axis=0 (columns) mean:", col_means)
# tensor([2.5000, 3.5000, 4.5000])
axis=0 (columns) mean: tensor([2.5000, 3.5000, 4.5000])

# row-wise mean (dim=1)
row_means = B.mean(dim=1)
print("axis=1 (rows) mean:", row_means) # tensor([2., 5.])
axis=1 (rows) mean: tensor([2., 5.])

```

```

import numpy as np

B = np.array([[1, 2, 3],
              [4, 5, 6]])

print(B.sum())           # 21
print(B.mean(), B.min(), B.max()) # 3.5 1 6
print(np.median(B), np.std(B))    # 3.5, 1.7078

# axis -----
print(B.sum(axis=0))      # column sums -> [5 7 9]
print(B.sum(axis=1))      # row sums   -> [ 6 15]

# column-wise mean (axis=0)
col_means = B.mean(axis=0)
print("axis=0 (columns) mean:", col_means)
# [2.5 3.5 4.5]
axis=0 (columns) mean: [2.5 3.5 4.5]

# row-wise mean (axis=1)
row_means = B.mean(axis=1)
print("axis=1 (rows) mean:", row_means) # [2. 5.]
axis=1 (rows) mean: [2. 5.]

```

torch.std() vs np.std()

PyTorch

```
import torch

A = torch.tensor([1, 2, 3, 4, 5, 6], dtype = float)

print(A.median(), A.std())           # tensor(3.), tensor(1.8708)
print(torch.median(A), torch.std(A)) # the same as the above
print(torch.std(A, unbiased=False))  # matches np.std
```

```
tensor(3., dtype=torch.float64) tensor(1.8708, dtype=torch.float64)
tensor(3., dtype=torch.float64) tensor(1.8708, dtype=torch.float64)
tensor(1.7078, dtype=torch.float64)
```

PyTorch computes the **sample** standard deviation:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$
$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)^2}$$
$$s = \sqrt{\frac{1}{5} (2 * (2.5^2 + 1.5^2 + 0.5^2))}$$

Numpy

```
import numpy as np

B = np.array([1, 2, 3, 4, 5, 6])

print(np.median(B), np.std(B)) # 3.5, 1.7078
```

3.5 1.707825127659933

NumPy computes the **population** standard deviation:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$
$$\sigma = \sqrt{\frac{1}{6} (2 * (2.5^2 + 1.5^2 + 0.5^2))}$$

Key Difference

The key difference lies in the denominator:

- NumPy: population(N)
- PyTorch (default): sample ($N - 1$)

reshape(), view()

PyTorch

```
import torch

C = torch.arange(12)      # tensor([0, 1, ..., 11])
print(C.reshape(3, 4))   # shape (3,4)
print(C.reshape(-1, 6))  # -1 lets PyTorch infer -> (2,6)
print(C.view(-1))        # flat view if possible
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
tensor([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11]])
```

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

c02_pytorch06.py

Numpy

```
import numpy as np

C = np.arange(12)        # [0..11]
print(C.reshape(3, 4))   # shape (3,4)
print(C.reshape(-1, 6))  # -1 lets NumPy infer -> (2,6)
print(C.ravel())         # flat view if possible
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

reshape(), view()

PyTorch

Create a contiguous tensor

```
A = torch.arange(12)
print("A:", A)
```

Reshape works fine

```
print("A.reshape(3,4):\n", A.reshape(3,4))
```

Make a non-contiguous tensor by transposing

```
B = A.reshape(3,4).t() # transpose
```

```
C = A.reshape(3,4)
```

```
print("\nB (transposed):\n", B)
```

```
print("Is B contiguous?", B.is_contiguous())
```

```
print("Is C contiguous?", C.is_contiguous())
```

```
tensor([[ 0,  4,  8],
        [ 1,  5,  9],
        [ 2,  6, 10],
        [ 3,  7, 11]])
```

```
Is B contiguous? False
```

```
Is C contiguous? True
```

view requires contiguous memory -> will raise an error

try:

```
print(B.view(12)) # RuntimeError
```

```
except RuntimeError as e:
```

```
print("B.view(12) failed:", e)
```

```
B.view(12) failed: view size is not compatible with input tensor's size and stride (at least one dimension spans across two contiguous subspaces). Use .reshape(...) instead.
```

reshape can handle non-contiguous tensors by copying

```
print("B.reshape(12):", B.reshape(12)) # works
```

```
B.reshape(12): tensor([ 0,  4,  8,  1,  5,  9,  2,  6, 10,  3,  7, 11])
```

- **view** → requires **contiguous** memory, otherwise it raises an error
- **reshape** → automatically creates a copy if needed and returns the result

reshape().t() vs flatten()

PyTorch

```
E = torch.arange(12).reshape(3,4).t() # non-contiguous
print("Is contiguous?", E.is_contiguous())
```

```
# flatten → always returns contiguous copy
print(E.flatten().is_contiguous()) # True
```

```
# view(-1) → only works if contiguous
try:
```

```
    print(E.view(-1))
except RuntimeError as e:
    print("view failed:", e)
```

```
Is contiguous? False
True
```

c02_pytorch06.py

```
view failed: view size is not compatible with input tensor's size and stride (at least one dimension spans a
cross two contiguous subspaces). Use .reshape(...) instead.
```

view()

```
import torch
```

```
t = torch.arange(12).view(3, 4)
print("reshaped (3x4):\n", t)
```

```
reshaped (3x4):
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

`B = A.reshape(3,4).t()` -> non-contiguous order in memory

Original contiguous buffer



0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

`reshape(3,4)` just change the view of memory: –only the *view* (shape/stride) changes, memory stays the same.

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

`B = A.reshape(3,4).t()` -> looks like a 4×3 matrix, but the elements are accessed in a non-contiguous order.

0	4	8	1	5	9	2	6	10	3	7	11
---	---	---	---	---	---	---	---	----	---	---	----

PyTorch

view

- Only if contiguous
- Fast (just metadata change)
- Raises error otherwise

reshape

- If the data is stored in a simple, contiguous block (e.g., created with `arange`, `zeros`, etc.),
→ reshape just changes the shape.
→ This is a view (very fast, no new memory).
- If the data is not contiguous (e.g., after transpose, slicing with steps, etc.),
→ reshape cannot simply reinterpret memory.
→ It will make a copy internally to give you the desired shape.

`torch.tensor([1., 2.])` is a 1-D tensor (=rank 1) whose shape is (2,).

- In both NumPy and PyTorch, a 1-D vector has no explicit row or column orientation.
- Depending on the operation, PyTorch (and NumPy) implicitly treats it as a row or as a column.

```
import torch

A = torch.tensor([[1., 2.],
                  [3., 4.]])

b = torch.tensor([1., 2.]) # define vector b

row = b[None, :]          # shape: (1, 2) ← row vector
col = b[:, None]          # shape: (2, 1) ← column vector

# Equivalent forms
row2 = b.reshape(1, -1)   # (1, 2)
col2 = b.reshape(-1, 1)   # (2, 1)

# Matrix multiplication
print(b @ A)              # (2,) @ (2,2) → (2,)   (behaves like a row in this context)
print(A @ b)              # (2,2) @ (2,) → (2,)   (behaves like a column in this context)

print(row @ A)            # (1,2) @ (2,2) → (1,2)
print(A @ col)            # (2,2) @ (2,1) → (2,1)
```

`row = b[None, :]`

`None` tells PyTorch to ‘add a new axis (dimension).’

```
tensor([ 7., 10.])
tensor([ 5., 11.])
tensor([[ 7., 10.]])
tensor([[ 5.],
        [11.]])
```



torch.stack()

torch.stack() → adds a new axis.

```
import torch
```

```
X = torch.tensor([1, 2, 3])
```

```
Y = torch.tensor([4, 5, 6])
```

```
print(torch.stack([X, Y], dim=0)) # [[1, 2, 3], tensor([[1, 2, 3],  
# [4, 5, 6]] [4, 5, 6]])
```

```
print(torch.stack([X, Y], dim=1)) # [[1, 4], tensor([[1, 4],  
# [2, 5], [2, 5],  
# [3, 6]] [3, 6]])
```

```
print(torch.vstack([X, Y])) # [[1, 2, 3], tensor([[1, 2, 3],  
# [4, 5, 6]] [4, 5, 6]])
```

```
print(torch.hstack([X, Y])) # [1, 2, 3, 4, 5, 6]  
tensor([1, 2, 3, 4, 5, 6])
```

```
import torch
```

```
X = torch.tensor([1, 2, 3])
```

```
Y = torch.tensor([4, 5, 6])
```

```
print(X.shape, Y.shape)
```

```
a = torch.stack([X, Y], dim=0)
```

```
b = torch.stack([X, Y], dim=1)
```

```
print(a.shape) # torch.Size([2, 3])
```

```
print(b.shape) # torch.Size([3, 2])
```

```
torch.Size([3]) torch.Size([3])  
torch.Size([2, 3])  
torch.Size([3, 2])
```

c02_pytorch08.py

Linear algebra

```
import torch
```

c02_pytorch09.py

```
A = torch.tensor([[2., 1.],  
                  [1., 3.]])  
b = torch.tensor([1., 2.])
```

```
# Solve  $Ax = b$ 
```

```
x = torch.linalg.solve(A, b)  
print("solution:", x)
```

```
solution: tensor([0.2000, 0.6000])
```

```
# Matrix multiply (dot product)
```

```
M = torch.tensor([[1, 2],  
                  [3, 4]], dtype=torch.float64)  
N = torch.tensor([[10, 20],  
                  [30, 40]], dtype=torch.float64)
```

```
tensor([[ 10.,  40.],  
        [ 90., 160.]], dtype=torch.float64)
```

```
print(M * N)           # element-wise multiplication  
print(M @ N)           # matrix multiplication  
print(torch.matmul(M, N)) # matrix multiplication (same as @)
```

```
tensor([[ 70., 100.],  
        [150., 220.]], dtype=torch.float64)
```

```
tensor([[ 70., 100.],  
        [150., 220.]], dtype=torch.float64)
```

```
# Eigenvalues & eigenvectors
```

```
eigvals, eigvecs = torch.linalg.eig(M)  
print("eigenvalues:", eigvals)  
print("eigenvectors:\n", eigvecs)
```

```
eigenvalues: tensor([-0.3723+0.j,  5.3723+0.j], dtype=torch.complex128)  
eigenvectors:  
tensor([[ -0.8246+0.j, -0.4160+0.j],  
        [ 0.5658+0.j, -0.9094+0.j]], dtype=torch.complex128)
```

Converting between Numpy and PyTorch Tensor

```
import torch
import numpy as np

np_array = np.array([1, 2, 3])

# Numpy -> Tensor
tensor_from_np = torch.from_numpy(np_array)
print("Tensor from numpy:", tensor_from_np)

# Tensor -> Numpy
back_to_np = tensor_from_np.numpy()
print("Back to numpy:", back_to_np)
```

```
Tensor from numpy: tensor([1, 2, 3])
Back to numpy: [1 2 3]
```

c02_pytorch10.py

Using GPU

```
# device (use GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

a = torch.tensor([1., 2.], device=device)
b = torch.tensor([1., 2.])
print(a)
print(b)
```

tensor([1., 2.], device='cuda:0')
tensor([1., 2.])

c02_pytorch10.py



CPU



GPU

torch.linspace(0, 1, N)

```
import torch
```

```
N = 5
```

```
x = torch.linspace(0, 1, N)
```

```
print(x)
```

```
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

```
y = torch.linspace(0, 1, N).unsqueeze(0)
```

```
print(y)
```

```
tensor([[0.0000, 0.2500, 0.5000, 0.7500, 1.0000]])
```

```
z = torch.linspace(0, 1, N).unsqueeze(1)
```

```
print(z)
```

```
tensor([[0.0000],  
        [0.2500],  
        [0.5000],  
        [0.7500],  
        [1.0000]])
```

```
print(x.shape, y.shape, z.shape)
```

```
torch.Size([5]) torch.Size([1, 5]) torch.Size([5, 1])
```

Optimizers in torch.optim



Optim.SGD

- The most **basic** Stochastic Gradient Descent.
- Supports options like momentum and nesterov

Optim.RMSProp

- Often used for training RNNs

Optim.Adam

- One of the most **widely used** optimizers.
- Based on **Adaptive Moment** Estimation.
- Combines the benefits of **momentum and RMSProp**.

Optim.AdamW

- A variant of Adam with a proper implementation of **weight decay**.
- Standard choice for **Transformer** models such as BERT.

Simplest Example (1): Linear Regression (1D Loss)



We aim to learn the parameters $a, b \in \mathbb{R}$ of a linear function:

$$y = f(x) = ax + b$$

Training Data

Given input samples $\{x_i\}_{i=1}^N$, we generate targets:

$$y_i = a^*x_i + b^* + \epsilon_i$$

where a^*, b^* are the true parameters and $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ is Gaussian noise.

Model Parameters

We initialize parameters randomly:

$$a \sim \mathcal{N}(0, 1), \quad b \sim \mathcal{N}(0, 1).$$

Prediction

For each input x_i , the prediction is:

$$\hat{y}_i = ax_i + b.$$

Simplest Example (1): Linear Regression (1D Loss)



Loss Function

We minimize the mean squared error (MSE):

$$\mathcal{L}(a, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2.$$

Optimization

Parameters are updated with gradient descent (SGD):

$$a \leftarrow a - \eta \frac{\partial \mathcal{L}}{\partial a}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b},$$

where $\eta > 0$ is the learning rate.

Result

After sufficient iterations, the learned parameters

$$(a, b) \approx (a^*, b^*)$$

converge to the true coefficients of the underlying linear function.

Simplest Example (1A): Linear Regression (1D Loss)

```
import torch
```

```
torch.manual_seed(0) # Reproducibility
```

c02_pytorchEx01A.py

```
# 1) Ground-truth (random) + data
```

```
a_star = torch.randn(1) # true slope
```

```
b_star = torch.randn(1) # true y-intercept
```

```
N = 200
```

```
x = torch.linspace(-5, 5, N).unsqueeze(1) # .unsqueeze(1) adds a new dimension at index 1 : making a column vector
```

```
noise_std = 0.5
```

```
y = a_star * x + b_star + noise_std * torch.randn(N, 1)
```

```
# 2) Parameters to learn (scalars with grad)
```

```
a = torch.randn(1, requires_grad=True)
```

```
b = torch.randn(1, requires_grad=True)
```

```
# 3) Optimizer (Adam tends to be steadier than plain SGD here)
```

```
opt = torch.optim.Adam([a, b], lr=0.05)
```

```
# 4) Training loop
```

```
for _ in range(3000): # more steps → steadier convergence
```

```
    y_hat = a * x + b
```

```
    loss = torch.mean((y_hat - y) ** 2)
```

```
    opt.zero_grad() # clear old gradients
```

```
    loss.backward() # compute  $\partial L / \partial a$ ,  $\partial L / \partial b$ 
```

```
    opt.step() # update a, b
```

```
print(f"True: a* = {a_star.item():.4f}, b* = {b_star.item():.4f}")
```

```
print(f"Learned: a = {a.item():.4f}, b = {b.item():.4f}")
```

```
print(f"MSE: {loss.item():.4f}")
```

```
True: a* = 1.5410, b* = -0.2934
Learned: a = 1.5372, b = -0.2266
MSE: 0.2349
```

Simplest Example (1B): Linear Regression (1D Loss)

With nn.Module

c02_pytorchEx01B.py

```
import torch
import torch.nn as nn

torch.manual_seed(0)  # Reproducibility

# 1) Ground-truth (random) + data
a_star, b_star = torch.randn(1), torch.randn(1)
N = 100
x = torch.linspace(-5, 5, N).unsqueeze(1)  # .unsqueeze(1) adds a new dimension at index 1 : making a column vector
y = a_star * x + b_star + 0.5 * torch.randn(N, 1)

# 2) Model, loss, optimizer
model = nn.Linear(1, 1)  # y ≈ ax + b (nn.Linear is a subclass of nn.Module), model is an object.
criterion = nn.MSELoss()  # MSELoss is a class and criterion is the instance of MSELoss class
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  # torch.optim.SGD is a class. Optimizer is an object.

# 3) Train
for _ in range(3000):  # more steps → steadier convergence
    y_hat = model(x)
    loss = criterion(y_hat, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

a, b = model.weight.item(), model.bias.item()
print(f"True: a* = {a_star.item():.4f}, b* = {b_star.item():.4f}")
print(f"Learned: a = {a:.4f}, b = {b:.4f}")
print(f"MSE: {loss.item():.4f}")
```

Let `criterion` be an instance of `nn.MSELoss`:

```
criterion = nn.MSELoss().
```

Callable Object via `__call__`

In Python, when we write

```
loss = criterion( $y_{\text{hat}}$ ,  $y$ ),
```

this does not call the constructor again. Instead, it invokes the special method

```
criterion.__call__( $y_{\text{hat}}$ ,  $y$ ).
```

Delegation to forward

For PyTorch modules, the method `__call__` internally delegates to the `forward` function:

```
criterion.__call__( $y_{\text{hat}}$ ,  $y$ )  $\longrightarrow$  criterion.forward( $y_{\text{hat}}$ ,  $y$ ).
```

Mean Squared Error Computation

Specifically, for `nn.MSELoss`, the forward method computes the mean squared error:

$$\mathcal{L}(y_{\text{hat}}, y) = \frac{1}{N} \sum_{i=1}^N (y_{\text{hat},i} - y_i)^2.$$

Summary

Thus,

```
loss = criterion( $y_{\text{hat}}$ ,  $y$ )
```

is equivalent to evaluating the MSE loss between the prediction y_{hat} and the target y .

IMPORTANT

Simple Example (2): Linear Regression



Let the input data matrix, weight vector, and target value be

$$X \in \mathbb{R}^{N \times d}, \quad w \in \mathbb{R}^{d \times 1}, \quad t \in \mathbb{R}^{N \times 1},$$

respectively. Here, $N = 5$ and $d = 3$ (5 samples, 3 features).

Predictions without bias:

$$\hat{y} = Xw.$$

Loss Function (MSE: Mean Squared Error)

$$L(w) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - t_i)^2 = \frac{1}{N} \|Xw - t\|_2^2.$$

If we use a scalar target c (broadcasted to all samples), then $t_i = c$ and:

$$L(w) = \frac{1}{N} \sum_{i=1}^N (x_i^\top w - c)^2.$$

target

$$t \in \mathbb{R}^{N \times 1}, \quad t = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

Gradients

$$\nabla_w L(w) = \frac{2}{N} X^\top (Xw - t),$$

Simple Example (2): Linear Regression

```
import torch

# device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# inputs
x = torch.randn(5, 3, device=device) # 5 samples, each with 3 features
target = torch.tensor([[0.5]]*5, device=device) # The shape of target vector (all 0.5) is (5,1).

# parameter to learn
w = torch.randn(3, 1, device=device, requires_grad=True) # random values are sampled from N(0,1)
# requires_grad=True tells autograd to track this tensor and compute gradients

# optimizer
optimizer = torch.optim.SGD([w], lr=0.1)

for step in range(20):
    pred = x @ w # The shape of pred is (5,1). @ does matrix multiplication.
    loss = ((pred - target)**2).mean() # MSE loss

    # autograd (= automatically computes gradients of tensors)
    optimizer.zero_grad() # Reset gradients from the previous step (set .grad to zero)
    loss.backward() # Compute gradients of loss (=d(loss)/dw) w.r.t parameters → stored in w.grad
    optimizer.step() # Update w using gradients: w = w - lr * w.grad

    if step % 5 == 0: # print every 5 steps
        print(f"Step {step:2d} | loss {loss.item():.4f} | w = {w.detach().T}")
        # .detach() breaks the tensor away from the autograd graph (so it won't track gradients).

print("Final w:", w.detach().T)
```

```
Step 0 | loss 0.8743 | w = tensor([[ 0.6639, -0.4362,  0.3074]], device='cuda:0')
Step 5 | loss 0.1398 | w = tensor([[ 0.2987, -0.5716,  0.0984]], device='cuda:0')
Step 10 | loss 0.0936 | w = tensor([[ 0.2158, -0.5652,  0.0095]], device='cuda:0')
Step 15 | loss 0.0859 | w = tensor([[ 0.1982, -0.5498, -0.0406]], device='cuda:0')
Final w: tensor([[ 0.1977, -0.5432, -0.0687]], device='cuda:0')
```

Simple Example (2): Linear Regression using torch.nn

```
import torch
import torch.nn as nn

# device (use GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# inputs
x = torch.randn(5, 3, device=device)          # 5 samples, 3 features each (shape: 5x3)
target = torch.full((5, 1), 0.5, device=device) # target values: constant 0.5 (shape: 5x1)

# model: y = x @ w, no bias (same as the original manual implementation)
model = nn.Linear(3, 1, bias=False).to(device)

# optimizer & loss function
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
loss_fn = nn.MSELoss()

for step in range(20):
    optimizer.zero_grad(set_to_none=True)      # reset gradients
    pred = model(x)                             # forward pass (output shape: 5x1)
    loss = loss_fn(pred, target)                # mean squared error loss
    loss.backward()                             # backpropagation
    optimizer.step()                           # parameter update

    if step % 5 == 0:
        with torch.no_grad():
            # model.weight has shape (1,3), so no need to transpose for printing
            print(f"Step {step:2d} | loss {loss.item():.4f} | w = {model.weight.detach()}")

with torch.no_grad():
    print("Final w:", model.weight.detach())
    print(target.shape)
```

c02_pytorchEx02B.py

with torch.no_grad(): → temporarily disables gradient tracking when you're only reading values (like model outputs or weights).

Model.weight → a learnable parameter in PyTorch. That means it is a torch.Tensor with **requires_grad=True** so that gradients can be computed during backpropagation.

```
Step  0 | loss 0.4216 | w = tensor([[ -0.4224,  0.5199, -0.5011]], device='cuda:0')
Step  5 | loss 0.2786 | w = tensor([[ -0.2865,  0.5620, -0.3276]], device='cuda:0')
Step 10 | loss 0.2443 | w = tensor([[ -0.2172,  0.5417, -0.2330]], device='cuda:0')
Step 15 | loss 0.2291 | w = tensor([[ -0.1780,  0.5005, -0.1719]], device='cuda:0')
Final w: tensor([[ -0.1569,  0.4636, -0.1352]], device='cuda:0')
torch.Size([5, 1])
```

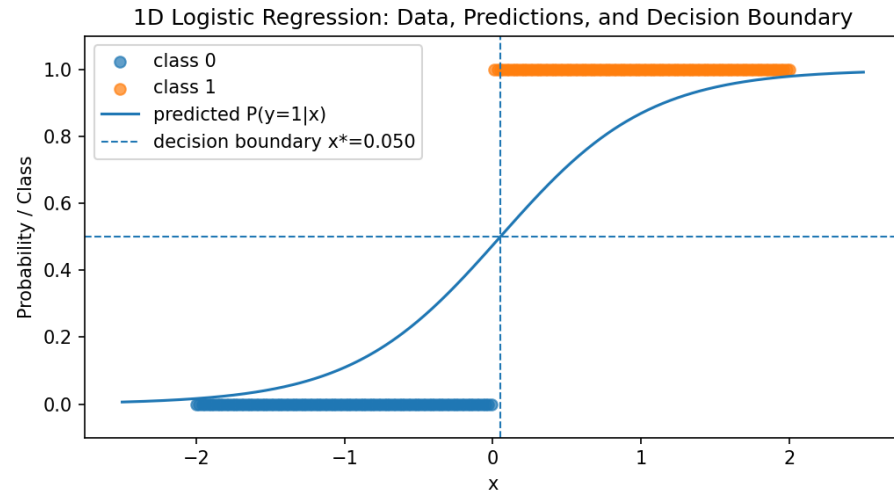
We use **.detach()** to safely look at the parameter values without gradient tracking.

Homework #1 (due date: 9.25 (Thu.))

c02_pytorchEx03.py does not take **bias** into account.

Please write a Pytorch code that performs linear regression with bias and send it to me via email (chocopie@kist.re.kr).

Simple Example (3): Binary Classification



1. Dataset (1D line)

We construct N (in our case, $N = 200$) evenly spaced points on a 1D line and assign binary labels via a step rule:

$$x_i \in \mathbb{R}, \quad x_i \text{ linspace in } [-2, 2], \quad y_i = \mathbf{1}\{x_i > 0\} \in \{0, 1\}, \quad i = 1, \dots, N.$$

In code: `x = torch.linspace(-2,2,N).unsqueeze(1)`, `y = (x[:,0] > 0).float()`.

2. Model (Logistic Regression)

A single linear (affine) function maps input $x \in \mathbb{R}$ to a scalar:

$$z_i = wx_i + b, \quad w, b \in \mathbb{R}.$$

The sigmoid (logistic) function gives the class probability

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{p}_i = \Pr(y_i = 1 \mid x_i) = \sigma(z_i) = \sigma(wx_i + b).$$

Simple Example (3): Binary Classification

3. Loss: Binary Cross-Entropy (with logits)

For a label $y_i \in \{0, 1\}$ and logit z_i ,

$$\ell(z_i, y_i) = -\left(y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))\right).$$

PyTorch's `BCEWithLogitsLoss` implements a numerically stable form. The empirical risk minimized during training is the mean loss:

$$\mathcal{L}(\boxed{w, b}) = \frac{1}{N} \sum_{i=1}^N \ell(wx_i + b, y_i).$$

4. Gradients (1D closed form)

Using $\frac{d}{dz} \ell(z, y) = \sigma(z) - y$, we get

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{N} \sum_{i=1}^N (\sigma(wx_i + b) - y_i) x_i, \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N} \sum_{i=1}^N (\sigma(wx_i + b) - y_i).$$

Simple Example (3): Binary Classification

```
import torch

# Toy dataset (1D line) Label = 1 if x > 0, else 0
N = 200
x = torch.linspace(-2, 2, N).unsqueeze(1)          # shape: (N,1)
y = (x[:,0] > 0).float()                          # step function

# Logistic regression model
model = torch.nn.Linear(1, 1)                     # w*x + b
criterion = torch.nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

# Training
for step in range(100):
    logits = model(x).squeeze(1)                  # raw scores
    loss = criterion(logits, y)                   # binary CE loss

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if step % 20 == 0:
        with torch.no_grad():
            preds = (logits > 0).float()           # threshold at 0
            acc = (preds == y).float().mean().item()
            print(f"step={step:3d}  loss={loss.item():.4f}  acc={acc:.3f}")
```

step= 0	loss=0.8361	acc=0.500
step= 20	loss=0.4480	acc=0.805
step= 40	loss=0.3181	acc=0.920
step= 60	loss=0.2595	acc=0.955
step= 80	loss=0.2259	acc=0.970

Thank you!

